In [3]:
```
# Sets : unordered collection of distinct hashable objects
# the elements of a set are immutable, but the set as a whole is mutable
# elements can be added or removed.
# Sets do not record element position or order of insertion.
# Sets do not support indexing, slicing, or other sequence-like behavior.

mySet1 = {'bca', 'btech', 'mca', 'mba'}
print(f"type: {type(mySet1)}, mySet1: {mySet1}")
```

```
type: <class 'set'>, mySet1: {'btech', 'mca', 'mba', 'bca'}
```

In [4]:
```
# duplicate values are ignored

mySet1 = {'bca', 'btech', 'mca','btech','mca'}
print(f"mySet1: {mySet1}")
```

```
mySet1: {'btech', 'mca', 'bca'}
```

In [5]:
```
# set() to create a new set

mySet1 = set(('mca', 'btech', 'bca'))
print(f"mySet1: {mySet1}")

mySet2 = set("students")
print(f"mySet2: {mySet2}")

mySet3 = set([1, 2, 3])
print(f"mySet3: {mySet3}")

mySet4 = set()
print(f"mySet4: {mySet4}")
```

```
mySet1: {'btech', 'mca', 'bca'}
mySet2: {'t', 'u', 'd', 'n', 's', 'e'}
mySet3: {1, 2, 3}
mySet4: set()
```

In [6]:
```
# to create an empty set we have to use set(), not {};
# {} creates an empty dictionary.
```

In [7]:
```
# len() - number of items in the set

mySet = {4, 2, 3, 3, 3, 1}
print(f"mySet: {mySet}, length: {len(mySet)}")
```

```
mySet: {1, 2, 3, 4}, length: 4
```

In [8]:
```
# cannot access items in a set by referring to an index
# but can loop through the set items

mySet = {'mca', 'btech', 'bca', 'mtech'}

for x in mySet:
  print(x)
```

```
btech
mca
mtech
bca
```

In [9]:
```
# can check if a specified value is present in a set, by using the in keyword

mySet = {'mca', 'btech', 'bca'}
```

```
print("mca" in mySet)
print("bba" in mySet)
print('mca' not in mySet)
```

```
True
False
False
```

In [10]:
```
x = set('abracadabra')
y = set('alacazam')
print(f"x: {x}") # unique letters in x
print(f"x - y : {x - y}") # letters in x but not in y
print(f"x | y : {x | y}") # letters in x or y or both
print(f"x & y : {x & y}") # letters in both x and y
print(f"x ^ y : {x ^ y}") # letters in x or y but not both
```

```
x: {'c', 'd', 'a', 'b', 'r'}
x - y : {'b', 'r', 'd'}
x | y : {'c', 'l', 'a', 'd', 'b', 'r', 'z', 'm'}
x & y : {'c', 'a'}
x ^ y : {'b', 'r', 'z', 'd', 'l', 'm'}
```

In [11]:
```
help(set.add)
```

```
Help on method_descriptor:

add(...)
    Add an element to a set.

    This has no effect if the element is already present.
```

In [12]:
```
# add set items
# add() method

mySet = set()
mySet.add(3)
mySet.add(1)
mySet.add(2)

print(mySet)
```

```
{1, 2, 3}
```

In [13]:
```
help(set.remove)
help(set.discard)
help(set.pop)
help(set.clear)
```

```
Help on method_descriptor:

remove(...)
    Remove an element from a set; it must be a member.

    If the element is not a member, raise a KeyError.

Help on method_descriptor:

discard(...)
    Remove an element from a set if it is a member.

    If the element is not a member, do nothing.

Help on method_descriptor:

pop(...)
    Remove and return an arbitrary set element.
    Raises KeyError if the set is empty.

Help on method_descriptor:

clear(...)
    Remove all elements from this set.
```

In [14]:
```python
# remove set items
# remove(), discard(), pop(), clear() methods

# remove() - removes the specified element from the set
# raises an error if the specified item does not exist

mySet = {4, 2, 3, 1}
mySet.remove(2)
# mySet.remove(5)
print(mySet)
```

```
{1, 3, 4}
```

In [15]:
```python
# discard() - removes the specified item from the set
# will not raise an error if the specified item does not exist

mySet = {4, 2, 3, 1}
mySet.discard(2)
# myset.discard(5)
print(mySet)
```

```
{1, 3, 4}
```

In [16]:
```python
# pop() can also be used to remove an element of a set
# as sets are unordered, which item will be popped can
# be known after pop() only

mySet = {2,1,4,3}
item = mySet.pop()
print(f"popped: {item}, mySet: {mySet}")
```

```
popped: 1, mySet: {2, 3, 4}
```

In [17]:
```python
# clear() method empties the set

mySet = {1,2,3,4}
mySet.clear()
print(f"after clear: {mySet}")
```

```
after clear: set()
```

In [18]:
```python
# del keyword will delete the set completely

mySet = {1,2,3,4}
del mySet
print(f"after del: {mySet}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[18], line 5
      3 mySet = {1,2,3,4}
      4 del mySet
----> 5 print(f"after del: {mySet}")

NameError: name 'mySet' is not defined
```

In [19]:
```python
print(dir(set))
```

```
['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update', 'union', 'update']
```

In [20]:
```python
help(set.union)
```

```
Help on method_descriptor:

union(...)
    Return the union of sets as a new set.

    (i.e. all elements that are in either set.)
```

In [21]:
```python
# union() method returns a set
# that contains all items from the original set,
# and all items from the specified set(s) (or any other iterable)

setA = {1, 3, 5}
setB = {2, 4, 6}
setC = {2, 3, 5, 7}

union_AB = setA.union(setB)
union_ABC = setA.union(setB, setC)

print(f"union_AB: {union_AB}")
print(f"union_ABC: {union_ABC}")
```

```
union_AB: {1, 2, 3, 4, 5, 6}
union_ABC: {1, 2, 3, 4, 5, 6, 7}
```

In [22]:
```python
help(set.update)
```

```
Help on method_descriptor:

update(...)
    Update a set with the union of itself and others.
```

In [23]:
```
# update() method updates the current set,
# by adding items from another set (or any other iterable)

setA = {1, 3, 5}
setB = {8, 6, 4}

setA.update(setB)

print(setA)
```

```
{1, 3, 4, 5, 6, 8}
```

In [24]:
```
help(set.intersection)
```

```
Help on method_descriptor:

intersection(...)
    Return the intersection of two sets as a new set.

    (i.e. all elements that are in both sets.)
```

In [25]:
```
# intersection() method returns a set
# that contains only items that exist in both sets,
# or in all the specifies sets.

setA = {1, 2, 3, 4, 5, 6, 7}
setB = {5, 6, 7, 8, 9}
setC = {3, 5, 7}

intersection_AB = setA.intersection(setB)
intersection_ABC = setA.intersection(setB, setC)

print(f"intersection_AB: {intersection_AB}")
print(f"intersection_ABC: {intersection_ABC}")
```

```
intersection_AB: {5, 6, 7}
intersection_ABC: {5, 7}
```

In [26]:
```
help(set.intersection_update)
```

```
Help on method_descriptor:

intersection_update(...)
    Update a set with the intersection of itself and another.
```

In [27]:
```
# intersection_update() method removes the items
# that is not present in both sets
# or in all the specified sets.

setA = {1, 2, 3, 4, 5, 6, 7}
setB = {5, 6, 7, 8, 9}

setA.intersection_update(setB)
print(f"intersection_AB: {setA}")
```

```
intersection_AB: {5, 6, 7}
```

In [28]:
```
setA = {1, 2, 3, 4, 5, 6, 7}
setB = {5, 6, 7, 8, 9}
setC = {3, 5, 7}
```

```
setA.intersection_update(setB, setC)
print(f"intersection_AB: {setA}")
```

```
intersection_AB: {5, 7}
```

In [29]:
```
help(set.difference)
```

```
Help on method_descriptor:

difference(...)
    Return the difference of two or more sets as a new set.

    (i.e. all elements that are in this set but not the others.)
```

In [30]:
```
# difference() method returns a set that contains
# items that exist only in the first set, and not in both sets.

setA = {1, 2, 3, 4, 5, 6, 7}
setB = {5, 6, 7, 8, 9}

difference_AB = setA.difference(setB)
difference_BA = setB.difference(setA)

print(f"difference_AB: {difference_AB}")
print(f"difference_BA: {difference_BA}")
```

```
difference_AB: {1, 2, 3, 4}
difference_BA: {8, 9}
```

In [31]:
```
help(set.symmetric_difference)
```

```
Help on method_descriptor:

symmetric_difference(...)
    Return the symmetric difference of two sets as a new set.

    (i.e. all elements that are in exactly one of the sets.)
```

In [32]:
```
# symmetric_difference() method returns a set that contains
# elements that are in exactly one of the sets
# i.e. excludes the items present in both sets

setA = {1, 2, 3, 4, 5, 6, 7}
setB = {5, 6, 7, 8, 9}

symDiff_AB = setA.symmetric_difference(setB)

print(f"sym_differ_AB: {symDiff_AB}")
```

```
sym_differ_AB: {1, 2, 3, 4, 8, 9}
```

In [33]:
```
help(set.symmetric_difference_update)
```

```
Help on method_descriptor:

symmetric_difference_update(...)
    Update a set with the symmetric difference of itself and another.
```

In [34]:
```
# symmetric_difference_update() method updates the original set
# by removing items that are present in both sets, and inserting the other items.

setA = {1, 2, 3, 4, 5, 6, 7}
setB = {5, 6, 7, 8, 9}
```

```
setA.symmetric_difference_update(setB)

print(f"sym_differ_upd_AB: {setA}")
```

```
sym_differ_upd_AB: {1, 2, 3, 4, 8, 9}
```

In [35]:

```
# The set type is mutable — the contents can be changed using
# methods like add() and remove(). Since it is mutable,
# it has no hash value and cannot be used as either a dictionary key
# or as an element of another set.

# The frozenset type is immutable and hashable — its contents
# cannot be altered after it is created;
# it can therefore be used as a dictionary key or as an element of another set.
```